

Conceptual Thinking

From Whatsapp to ...
...objects and classes

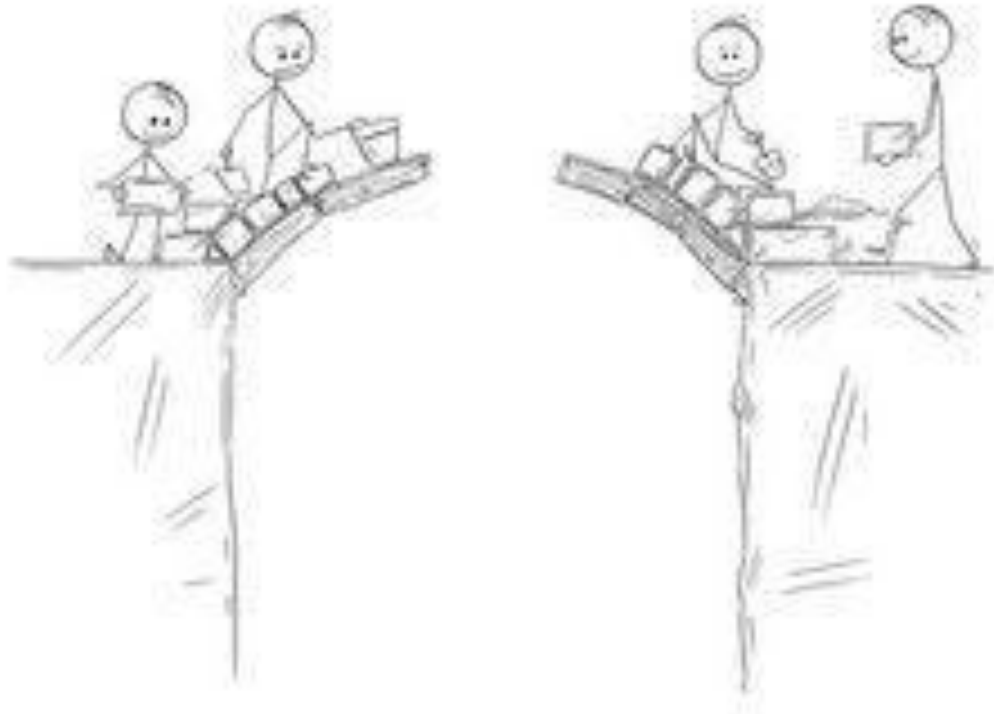
Contents



- 1. Conceptual Thinking: The BIG Picture**
- 2. Software Development LifeCycle (SDLC)**
- 3. Object Oriented Analysis & Design (OOAD)**
- 4. Glossary**
- 5. Unified Modelling Language (UML)**
- 6. Group assignment**

Conceptual Thinking: The BIG Picture

Why Conceptual Thinking?



Why Conceptual Thinking?



How the customer explained it



How the Project Leader understood it



How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



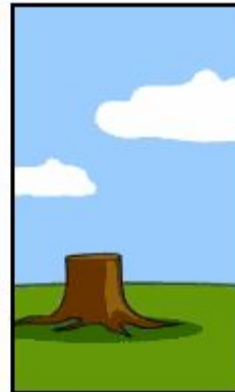
How the project was documented



What operations installed



How the customer was billed



How it was supported



What the customer really needed

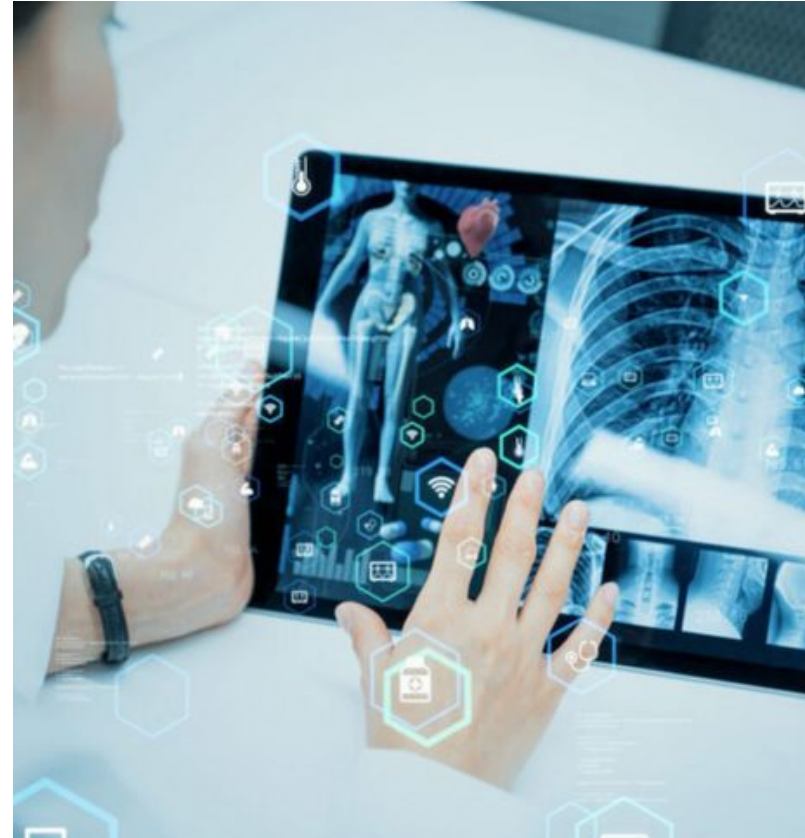
Solving problems

It's not just about coding; it's about understanding user needs, creating solutions and making sure they keep working.



Healthcare solution

Helping doctors monitor patients remotely, track health data and provide remote care.



Online Learning

**Like this course-
helps us stay connected,
learn and access resources
from any location.**



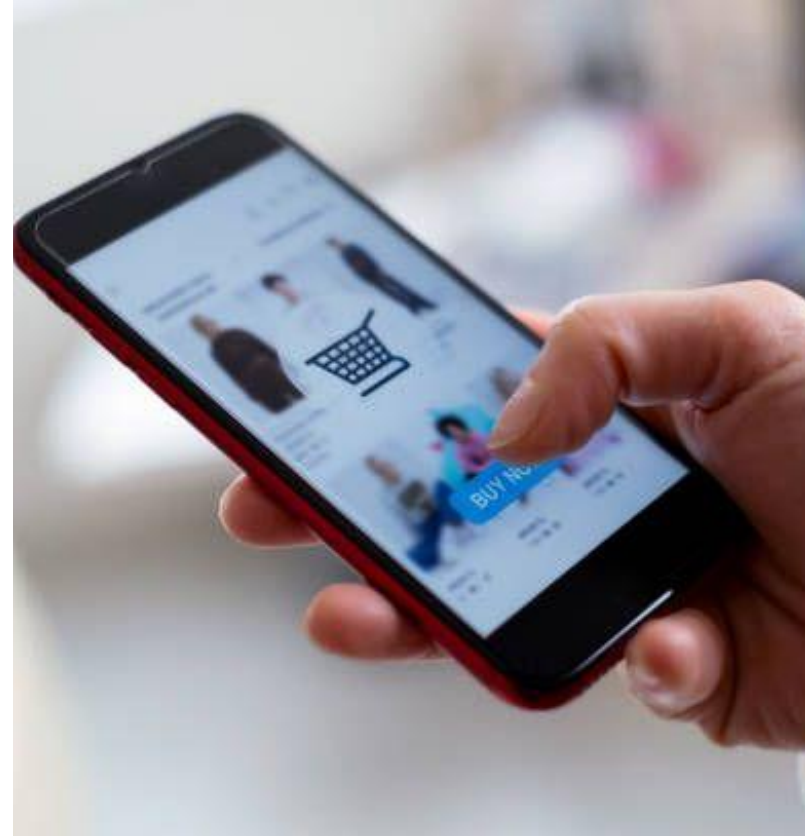
Emergency Services Apps

Help firefighters respond to emergencies by getting real-time data on locations and risks.



Online shopping

- **Online shopping: solving the problem of shopping at home.**



Banking solutions

Banking with apps: making transactions easier and faster.



Transport

Ridesharing services: solving transport challenges.



Identify a problem yourself

- **In groups, think of a problem you regularly encounter on campus or in your daily life.**
- **Discuss how software can help solve that problem.**



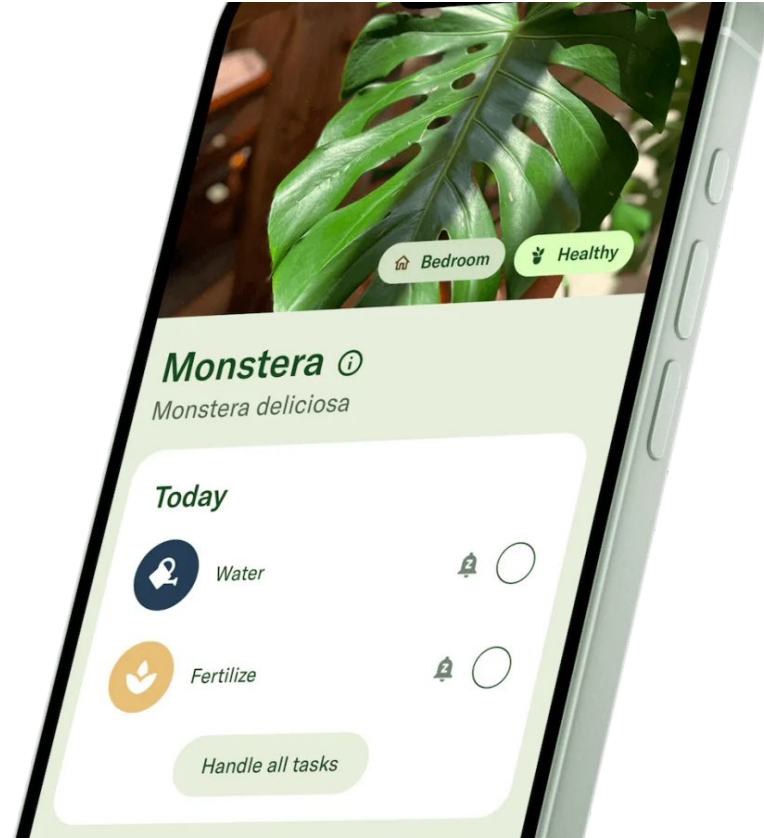
**I alone cannot change
the world, but I can cast
a stone across the
waters to create many
ripples.**

MOTHER TERESA

EVERYDAYPOWER

Example!

- **Problem: Forgetting to water your plants.**
- **Solution: An app that sends reminders to water your plants based on weather data.**



Share via Teams!





Who
recognises
this symbol?



Situation outline: You have sent a message via WhatsApp to a friend or a group chat. After a while, you notice that you have not received a response. You start to wonder what is going on and look for information to better understand the situation.

Q: First stage, right after sending the message. What do you want to know about it?



Message information:

- Has the message been delivered?
(Two grey ticks)
- Has the message been read (Two blue tick marks)
- When was the message delivered and/or read (Time stamp of delivery/read)?



Situation outline: You have sent a message via WhatsApp to a friend or a group chat. After a while, you notice that you have not received a response. You start wondering what is going on and you look for information to better understand the situation.

Question: later stage, Suppose there is still no response, even hours later. What do you want to know about it?



User information:

- Last time online: Has the person been online recently? This can help you understand if they might have missed or ignored your message.
- Name and Profile Picture: Sometimes you may want to check if you sent the message to the right person (especially if you have multiple contacts with the same first name).
- User status: Is the person active or inactive (e.g. status "Available", "Do not disturb", etc.)?



Group information:

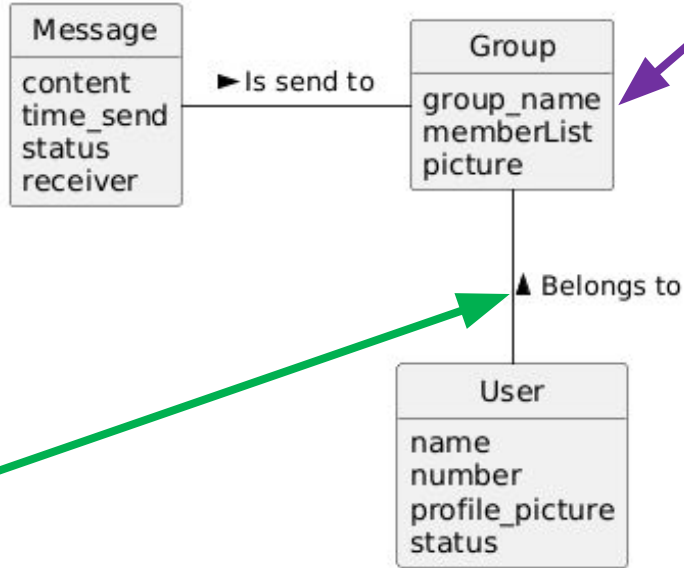
- Which members have read the message? Especially in group chats, it is useful to see who in the group has read the message (and who has not).
- Group name: Sometimes you send a message to several groups, and you want to check if you sent the message to the right group.
- Who is in the group? You might even want to check the group members to see if everyone you expected is participating in the chat.



CLASS: A class is a blueprint or template for objects.

ASSOCIATION: a relationship between two or more **CLASSES** that describes how they are connected.

Class Diagram



ATTRIBUTE: what information do we keep by class

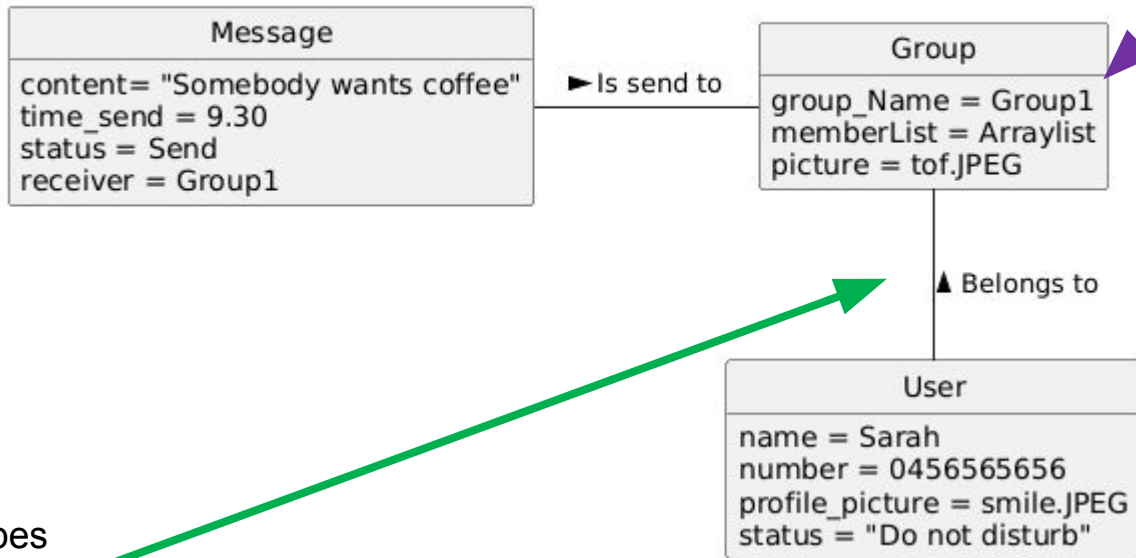


Object Diagram

ATTRIBUTE VALUES: what information do we store about the attribute

OBJECT: completed blueprint of a class.

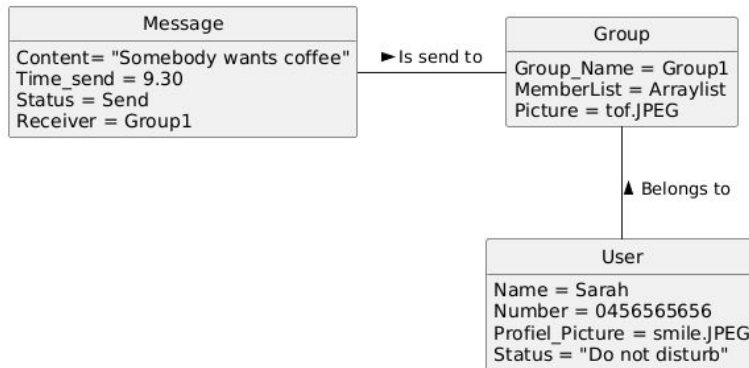
LINK: a relationship between two or more **OBJECTS** that describes how they are connected.



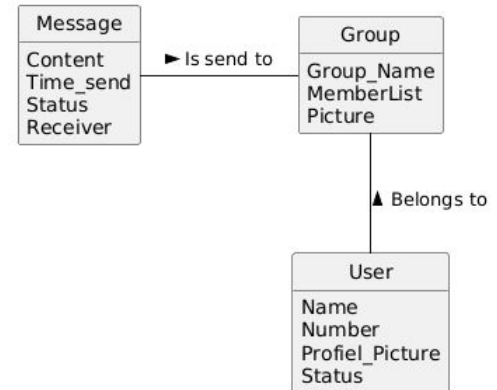
Summary

How many objects?
How many Classes?
How many attributes?
How many links?
Which diagram : Left - Right?
How many associations?

Object Diagram



Class Diagram



Exercise 1.1 : Find the classes, attributes and associations

Tool: Draw.io

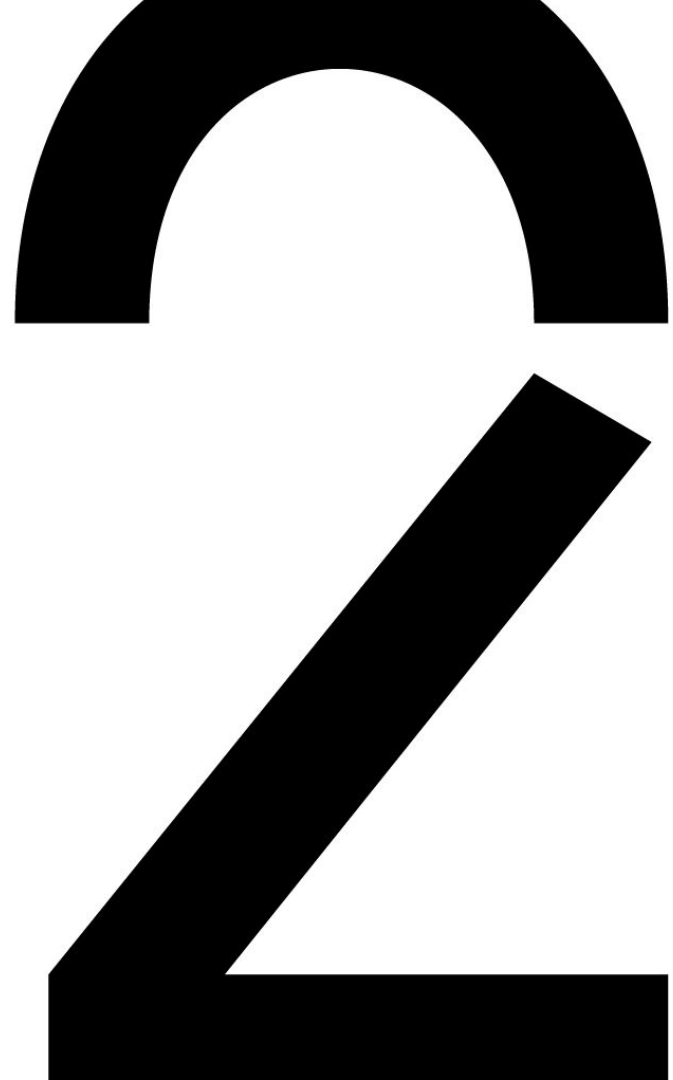


Situation outline: You choose a movie on Netflix, and the next day you log in again to watch something new. Netflix makes some recommendations based on your viewing behaviour, and you can also check your viewing history to see which films or series you have watched recently.

Questions:

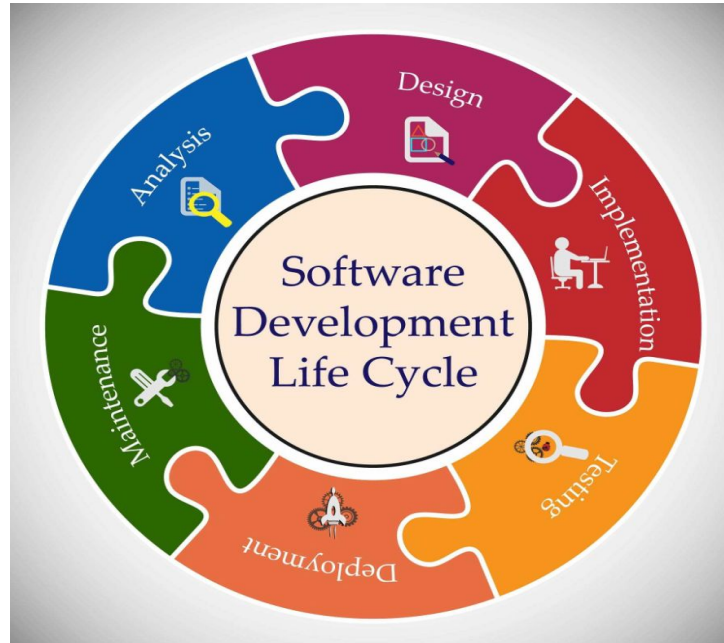
- Before you choose the film. What do you want to know about it?
- How does Netflix make recommendations?
- How can I watch Netflix and my sister too?

Software Development LifeCycle (SDLC)



The software development life cycle (SDLC)

SDLC (Software Development Life Cycle) is the **structured process** followed when developing software. It describes all the steps from idea to delivery and maintenance.



Source: [The software development life cycle - the complete guide](#)

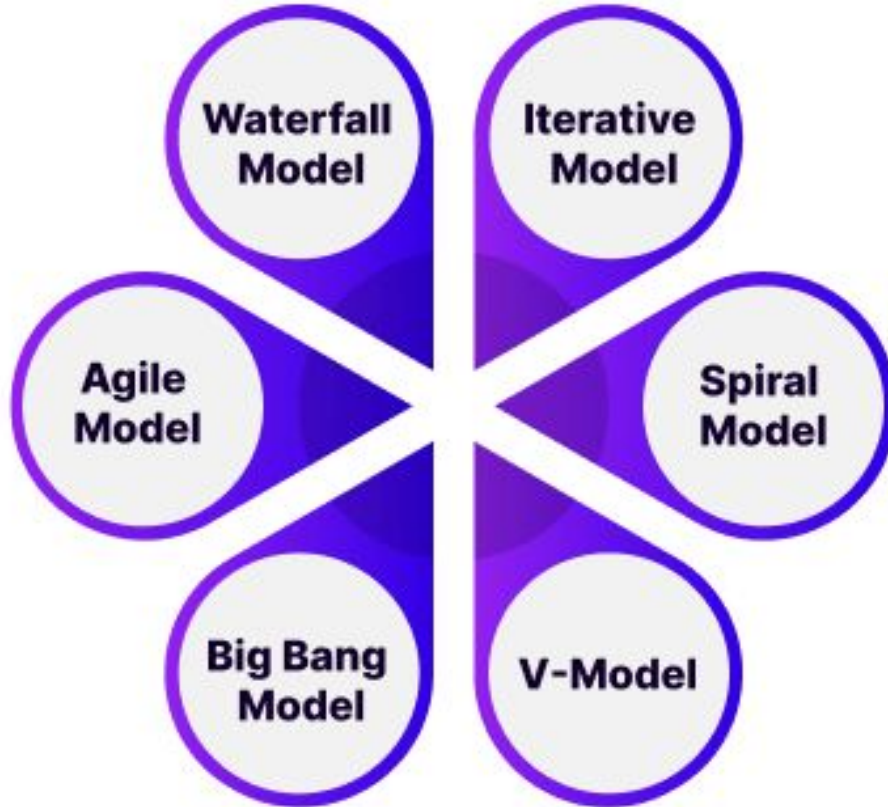
Phases of the SDLC

Phase	Explanation: Each phase builds on the previous one and contributes to a stable and working software solution.
1. Analysis	Understand what the problem is, collect requirements from users/customers.
2. Design	Design the technical solution (e.g. UML diagrams, system architecture).
3. Implementation	Programming the software according to the design.
4. Testing	Testing for errors and checking that the software works as expected.
5. Roll out	The software is installed or launched for use by end users.
6. Maintenance	Ongoing support: bug fixes, updates and improvements after delivery.

Why SDLC is important

- **Efficiency:** A plan ensures that software is built systematically
- **Quality:** Testing and refinement, ensures that the final product meets user **requirements** and is free of **critical bugs**.
- **Cost control:** Careful planning reduces risks/costs of expensive rework.
- **Clear communication:** documentation informs stakeholders

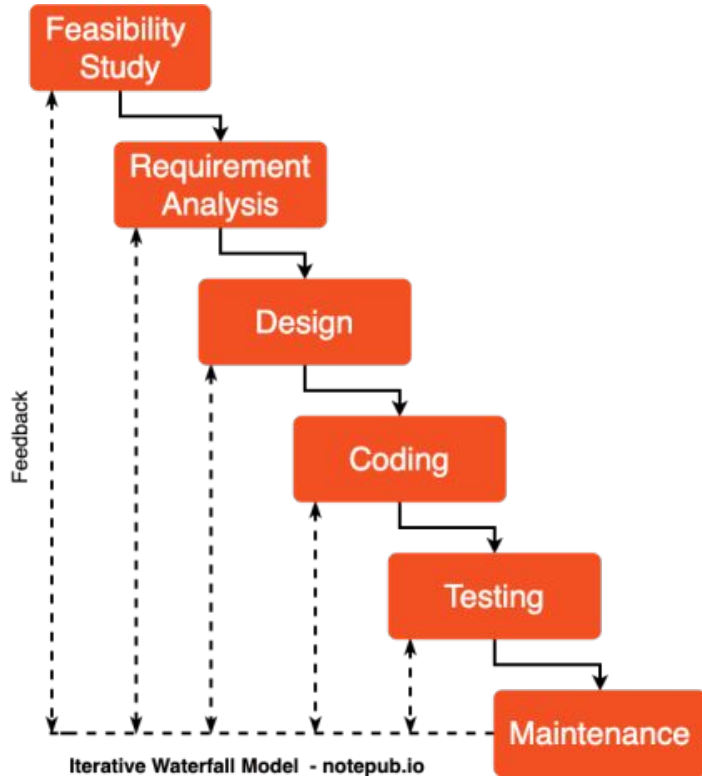
The software development life cycle (SDLC)



Models = ways to implement SDLC, such as:

- Waterfall = Phases are completed linearly and sequentially, without revisiting.
- Agile = Focused on flexibility and collaboration, with regular working deliveries.
- V-model = Each development phase has a corresponding test phase (verification vs validation).
- Spiral = Combines iterations with risk management, especially for complex projects.
- Big Bang = Starts without planning, all development happens at once - chaotic.
- Iterative = Software is developed incrementally in small versions (iterations).

The software development life cycle (SDLC)

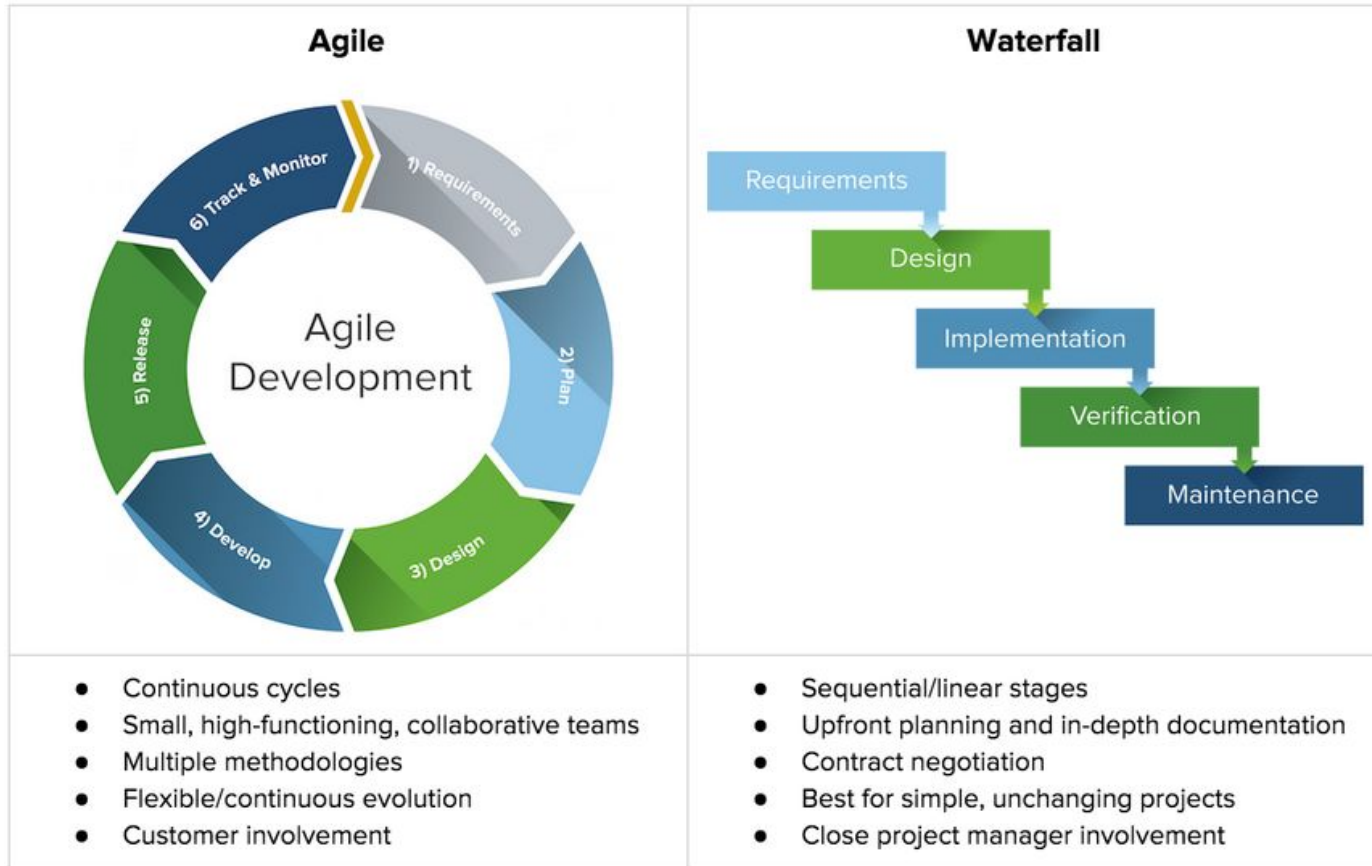


The software development life cycle (SDLC)

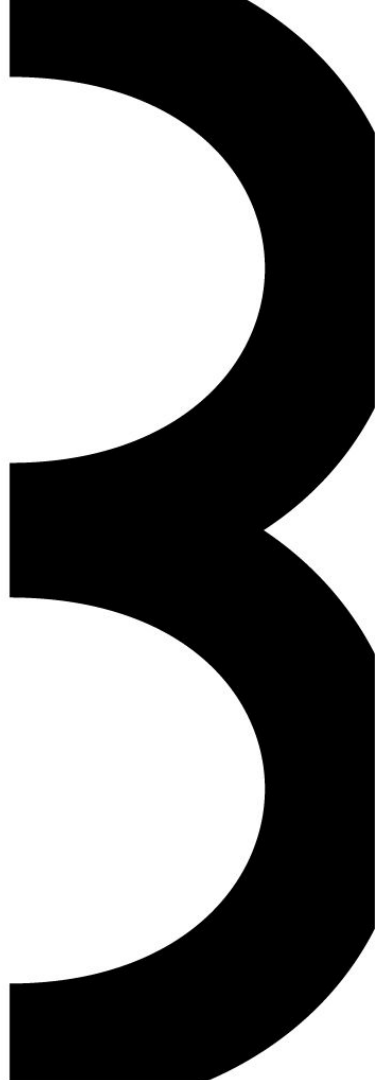
AGILE



Agile vs Waterfall



UML



UML (Unified Modelling Language)

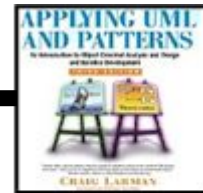
1. What is UML?

- UML is a visual modelling language for designing and documenting software.
- Mainly used in Object-Oriented Design.
- UML makes abstract systems visually understandable.

2. The usefulness of UML:

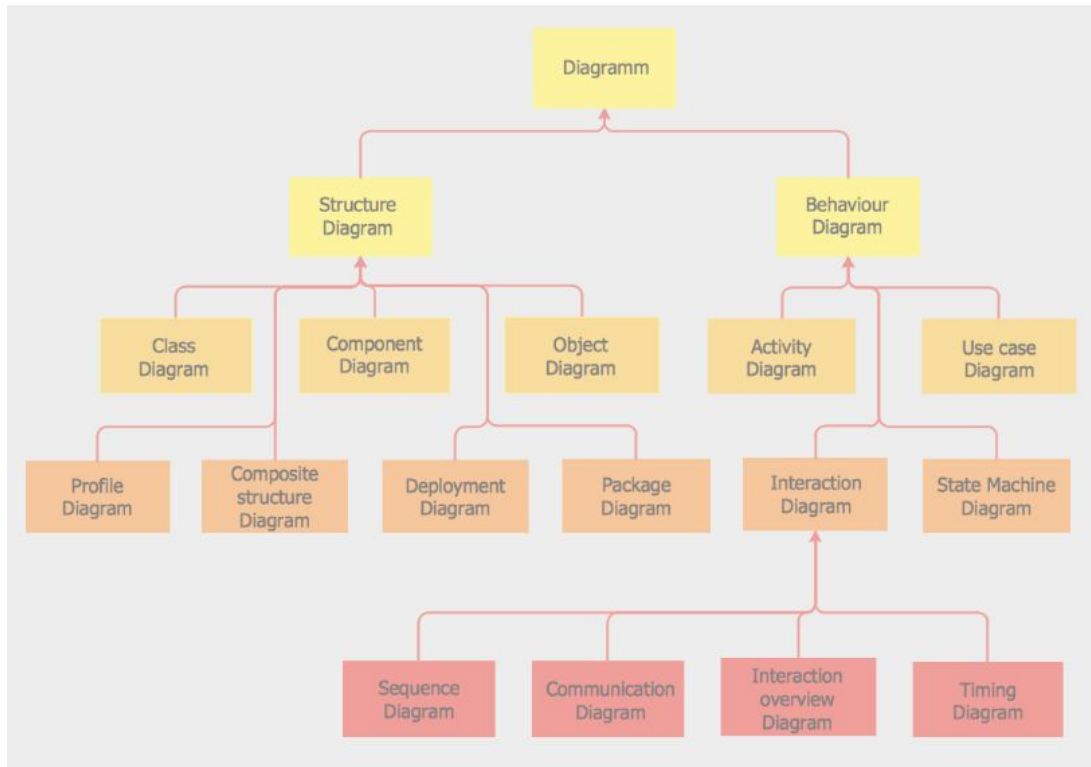
- UML helps to structure concepts visually.
- Ensures shared understanding between developers, analysts and customers.
- UML is language-independent (not limited to Java or Python).
Basis for translating conceptual models into code.

Diagrams with UML notation



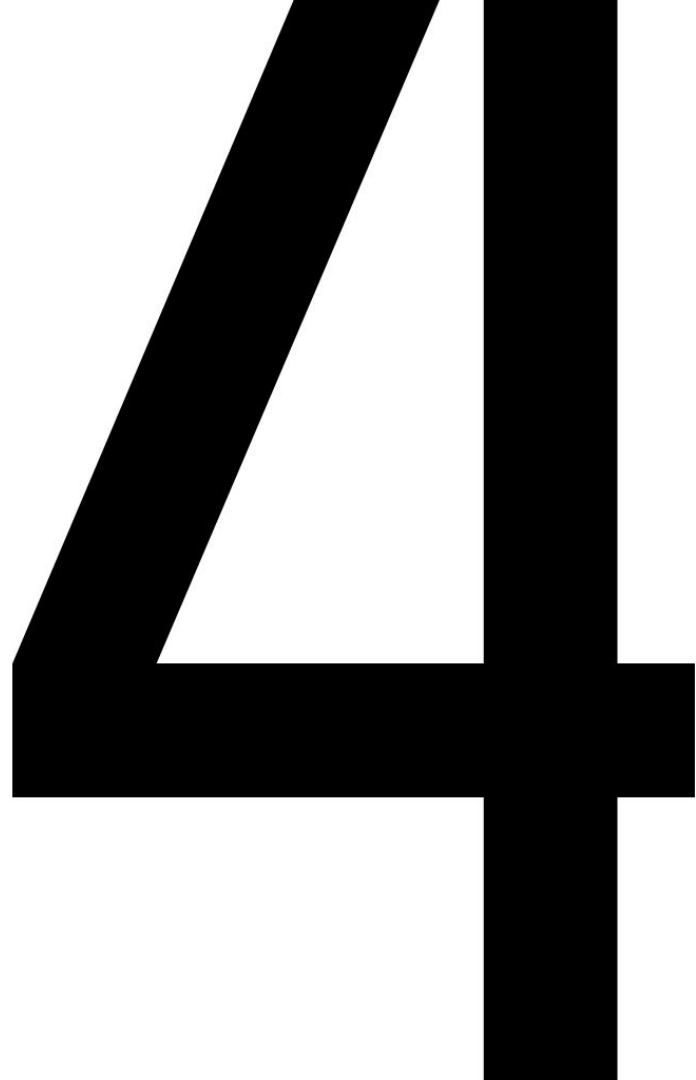
10.4

UML (Unified Modelling Language) defines **14 official diagram types**, divided into **2 major categories**:



Source: [UML Diagram Types List](#)

Object Oriented Analysis & Design (OOAD)



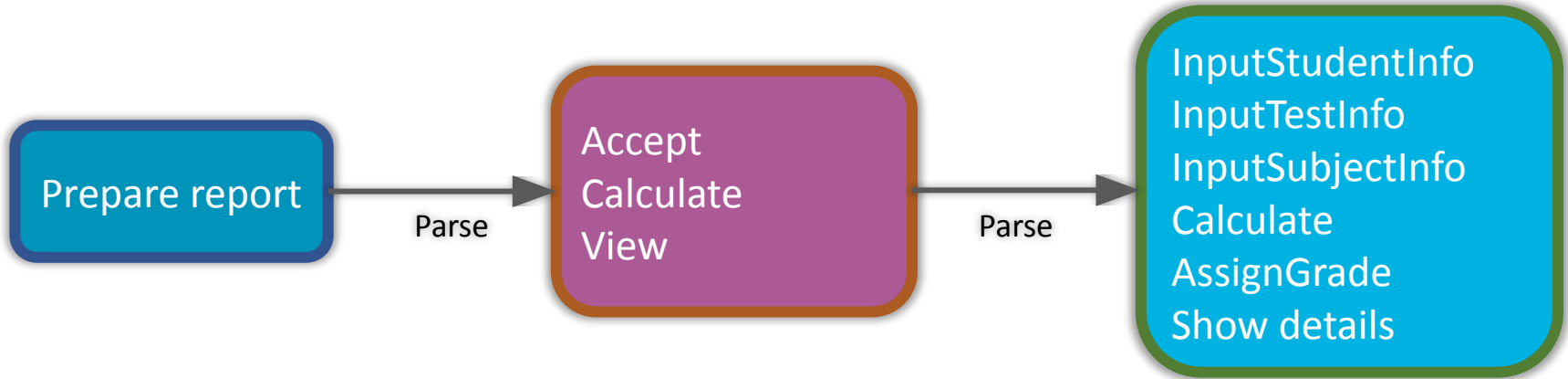
Resolving complexity

- Humans have a **built-in mechanism** for dealing with complexity
- Decompose each component **into smaller parts** until each **makes sense individually**
- Finally, **integrate** the parts to **build** the final system
- The same principle is applied to programming languages

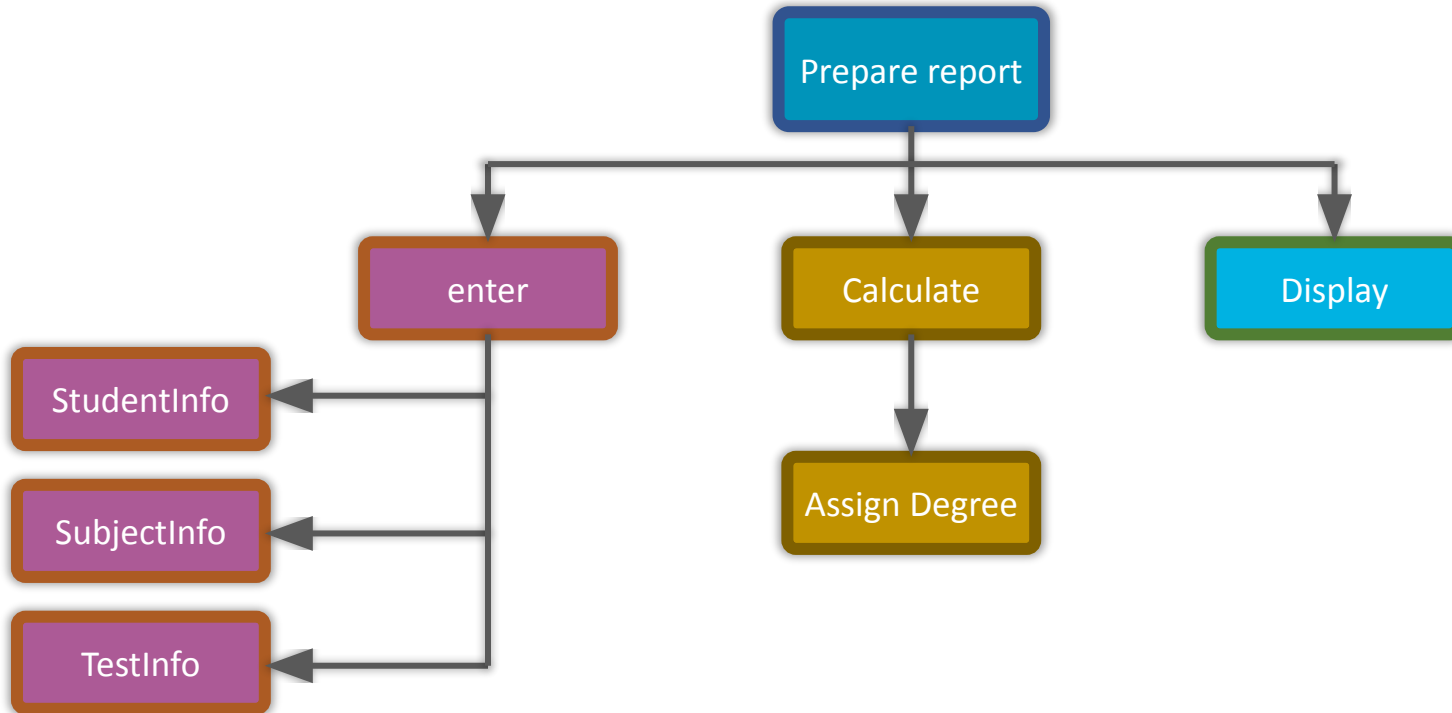
Sample

- Write a software to help teachers create a report for their students
- The software will
 - Accept student data (name, class, role number)
 - Accept test data (semester/unit)
 - Accept subject name and score
 - Calculate total score
 - Calculate average score
 - Assign grade
 - Show lowest and highest scoring subjects (based on a certain threshold)

Dissect



Execution

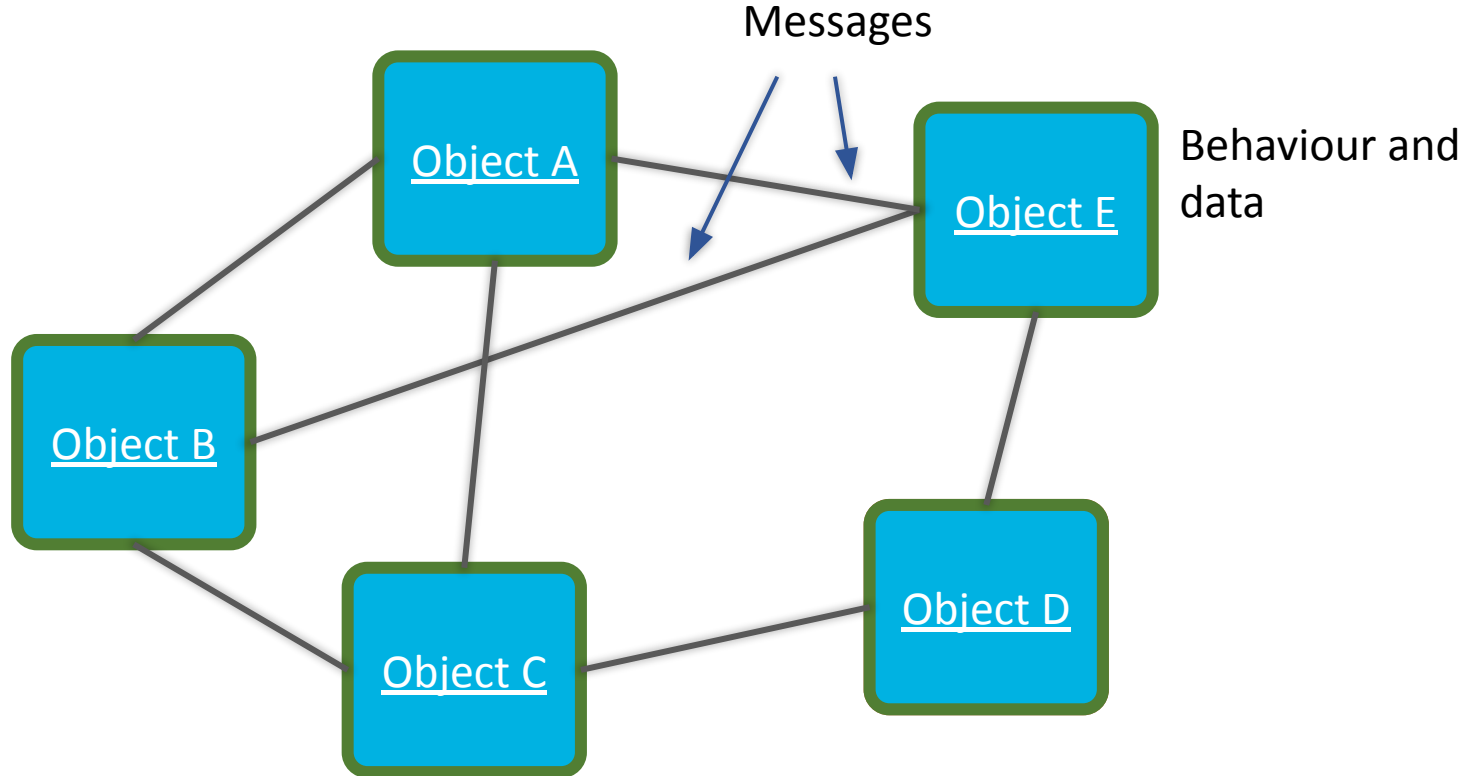


What is Object-oriented decomposition?

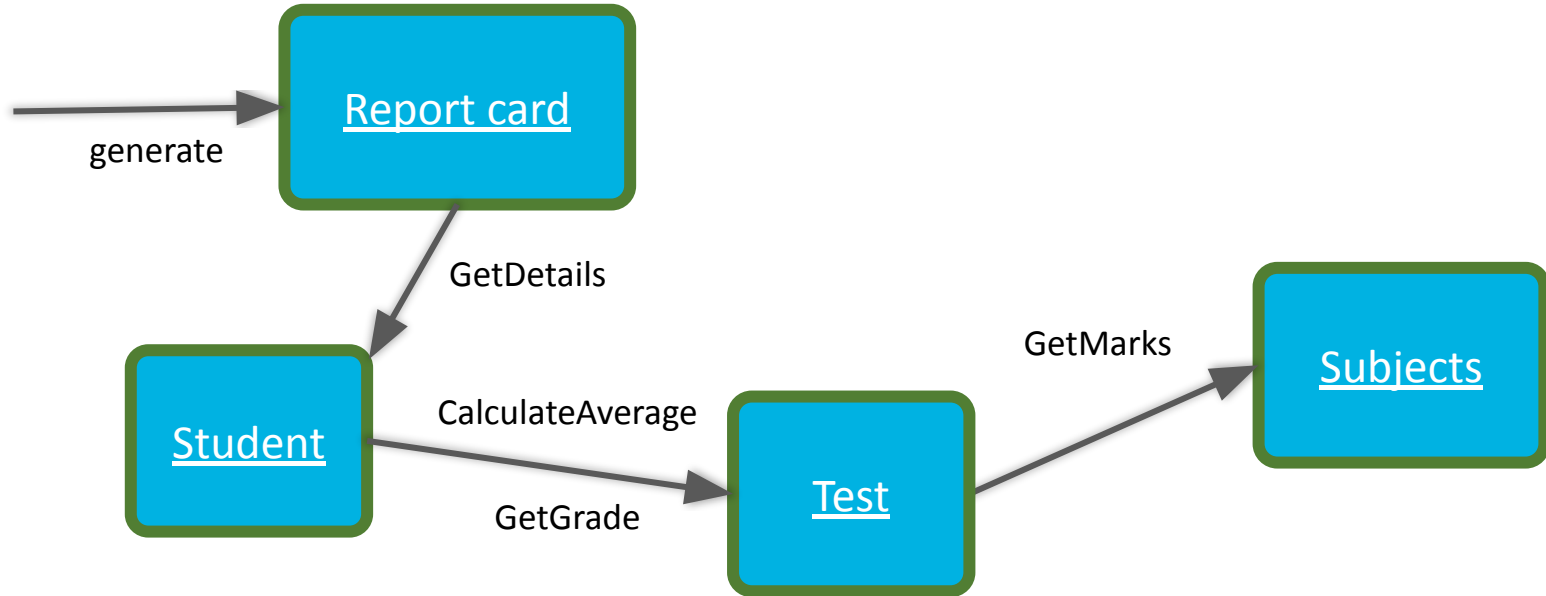
- Fundamental **concept** in Object-Oriented Analysis and Design (OOAD)
- Process of **breaking down** a **complex** system or problem into **smaller, manageable** objects or components



Object-oriented decomposition



Object-oriented decomposition



Benefits of decomposition

- Follows **separation of concerns**
 - each entity has its own **responsibility**
 - Addresses complexity through **organisation**
- System consists of objects representing **real-life entities**
- Closely matches **real** problems
- Data are of **great importance**
 - Part of the object, **not visible** from outside
- Enables **generalisation** of objects
 - Promotes **reuse** of common functionality
 - Leads to **smaller** systems
- Evolves **incrementally** over a period of time



Advantages

This image shows the essence of object-oriented design as "Lego blocks":

- In the centre, you see a red LEGO cube: this symbolises a reusable class or component.
- On the left is a house, on the right a car, and the same red cubes are clearly used in both.
- This illustrates that one well-designed cube (class) is reusable in different contexts.
 - reusable, maintainable and flexible.

In short: build with reusable building blocks, then you can make more with less.



Question...

What is an **object**?

Object definition (*-Smith and Tockey)

"An **object** represents an **individual, identifiable** item, **unit** or **entity**, real or **abstract**, with a well-defined **role** in the **problem domain**."

- Models a part of **reality**
- **Exists** in space and time
- Can be **invented** as a result of a design process
- Has well-defined **behaviour** and a clear **purpose**
- **Interacts** with other objects to provide **higher-level** behaviour

Object characteristics

- An object has the following characteristics:
 - **condition**
 - **identity**
 - **behaviour**
- The structure and behaviour of similar objects are specified in a **class**
 - an object is an **instance** of such a **class**
 - a class is the **blueprint** for objects
- forms the **building block** of an application

Object state

- **Properties** and their **values** constitute the **state** of an object
- Appear as **attributes** of an object
 - fields, member variables, etc.
- Can be of two **types**
 - **static** - fundamental attributes that **do not change**
 - **dynamic** - attributes that **change** as a result of **an operation** performed on the object

Object status: Example - Speaker system

Static

Power
Number of
speakers (2, 2.1,
4.1, etc.)
Connectivity
Colour
Brand
Serial no.



Dynamic

Volume
Treble
Bass

Object status: Example - Mobile phone

Static

CPU
Storage size
Screen size
Supported bands
OS
IMEI number



Dynamic

State (standby, busy)
Remaining memory
Battery level
Signal strength

Object identity

- **Feature** that makes an object **unique** and gives it **individuality**
- Is represented by **one or more attributes** of the object
 - **Address** of an object can in some cases be used to represent its identity
- Helps **identify** objects in a system
- Examples
 - Voice system - serial number
 - Mobile phone - IMEI number
 - Person - social security number

Why is object identity important?

- **Tracking:** Identity enables systems to track objects over time, making it possible to recognise the same object despite state changes.
- **Persistence:** For systems that store objects (e.g. in databases), identity ensures that objects can be retrieved and updated correctly.
- **Communication between objects:** When objects communicate with each other, their identity ensures that the system can guarantee that the message is sent to the correct object, even if others look the same.

Object behaviour

- How an object **acts** and **reacts** during **interactions** within a system.
- This behaviour is **defined** by its attributes (**data**) and operations (**methods** or functions).
 - For example, a "Speaker" object may have attributes such as volumeLevel, bassLevel, or trebleLevel.
 - Operations for the "Speaker" object may be ChangeVolume(), AdjustBass() or EqualiseLoudness().
- An important aspect of an object's responsibility.
- Operations internally change the state of the object.

Examples Object behaviour

Speaker system: When you execute the **changeVolume(20)** method, the Speaker object **changes** its internal **volumeLevel** **attribute**. This status change determines the **behaviour** of the Speaker - **its** volume output.



Examples Object Behaviour (Mobile phone)

When you call the **dial()** **function**, the mobile phone object interacts with its **internal state** (e.g. current signal strength) and **external** systems (e.g. network) to initiate a call. The object's **response** to this operation **reflects** its **behaviour**.



Object model

- The object model forms the basis of three main concepts
 - **Object-oriented analysis (OOA)**
 - **Object-oriented design (OOD)**
 - **Object-oriented programming (OOP) (Programming 1)**

Object-oriented analysis

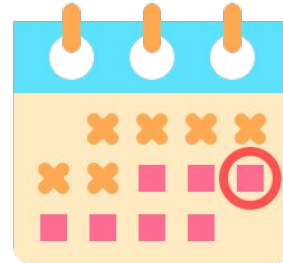
- Process of **understanding** and **modelling** the **problem domain**
 - Identifying the system requirements
 - And the objects that exist in that domain
- The goal of OOA is to develop a conceptual model that captures the following
 - Real-world entities (**Objects**)
 - **Attributes**
 - **Relationships** between **entities**
 - **Actions** or **behaviours** that an object can perform (**Methods**)
 - **Interactions** (Use Cases) between system and users (**actors**)

OO *Analysis* *library*

- Member requests a book from the library
- Librarian searches for the book
- If the book is found, it is published
 - The database is updated with the published book
- If the book is not available, the request is placed in a queue
- If the book is not found, it can be ordered

Imagine you are designing a system for a hospital

- You would need to model **patients, doctors** and **appointments**.
- OOAD helps us break down **complex** systems into **simple objects** that we can **easily** model and **work** with.



What is object-oriented analysis?

We look at **real** objects,
such as

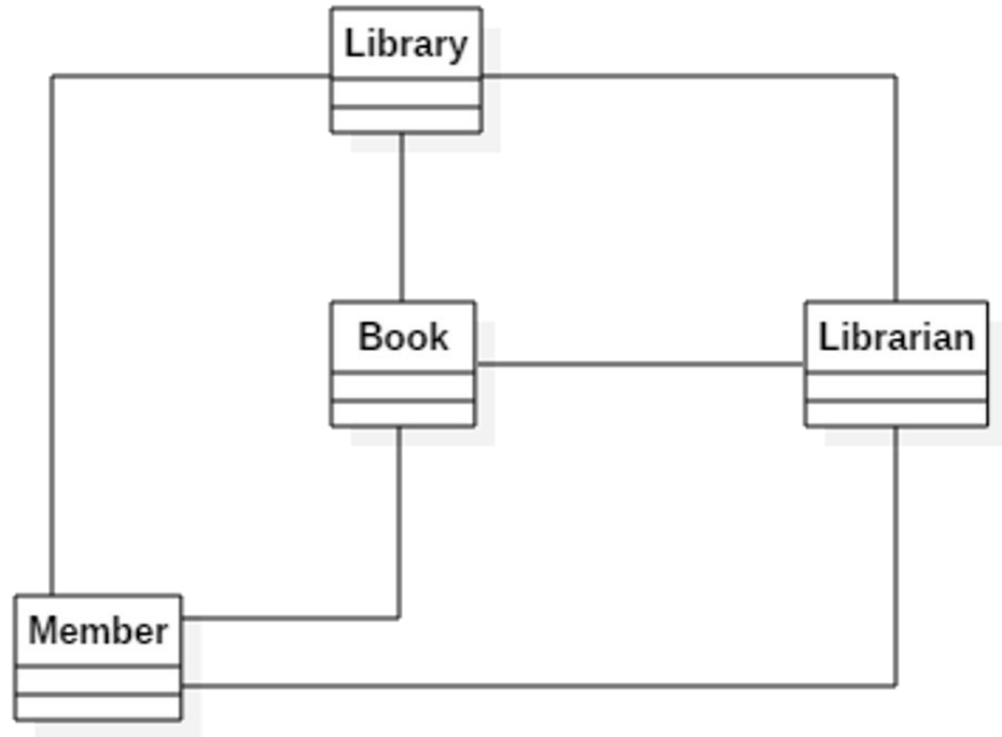
- a **car**,
- a **student**,
- or a doctor



Object-oriented design (OOD)

- Builds on the OOA phase
- Transforms the conceptual model (OOA) into a logical design
- Focuses on defining how
 - The system will be structured
 - The objects interact to achieve the goals of the system
- **Key concepts**
 - **Classes:** Define the **blueprint** for creating **objects**
 - **Class diagrams:** Visual representations of classes
 - **Composition** and **aggregation:** These represent "**part-of**" relationships between objects

OO Design library

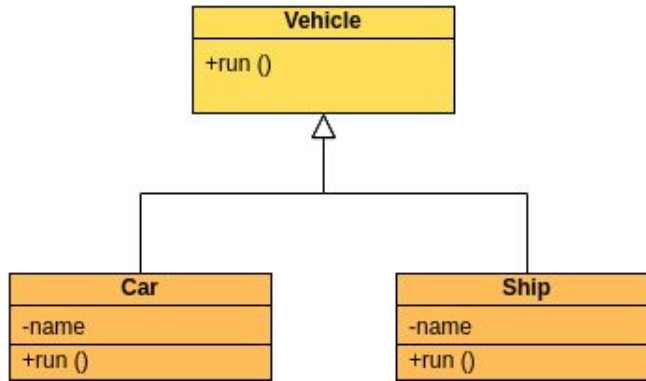


What is object-oriented design?

We look at **real** objects,
such as

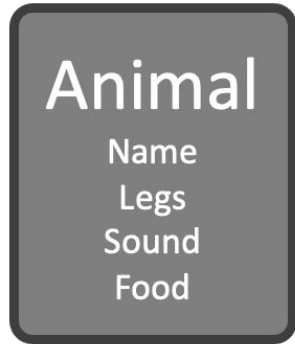


- a **car**,
 - a **student**,
 - or a doctor
-
- and model them in **software**.

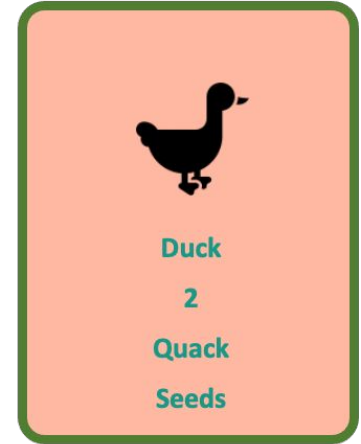


What is a class?

- A **class** is a **type** or **blueprint** of an **entity**
- Its **instance** is called **an object**
- An **object** of a class represents a specific **type**

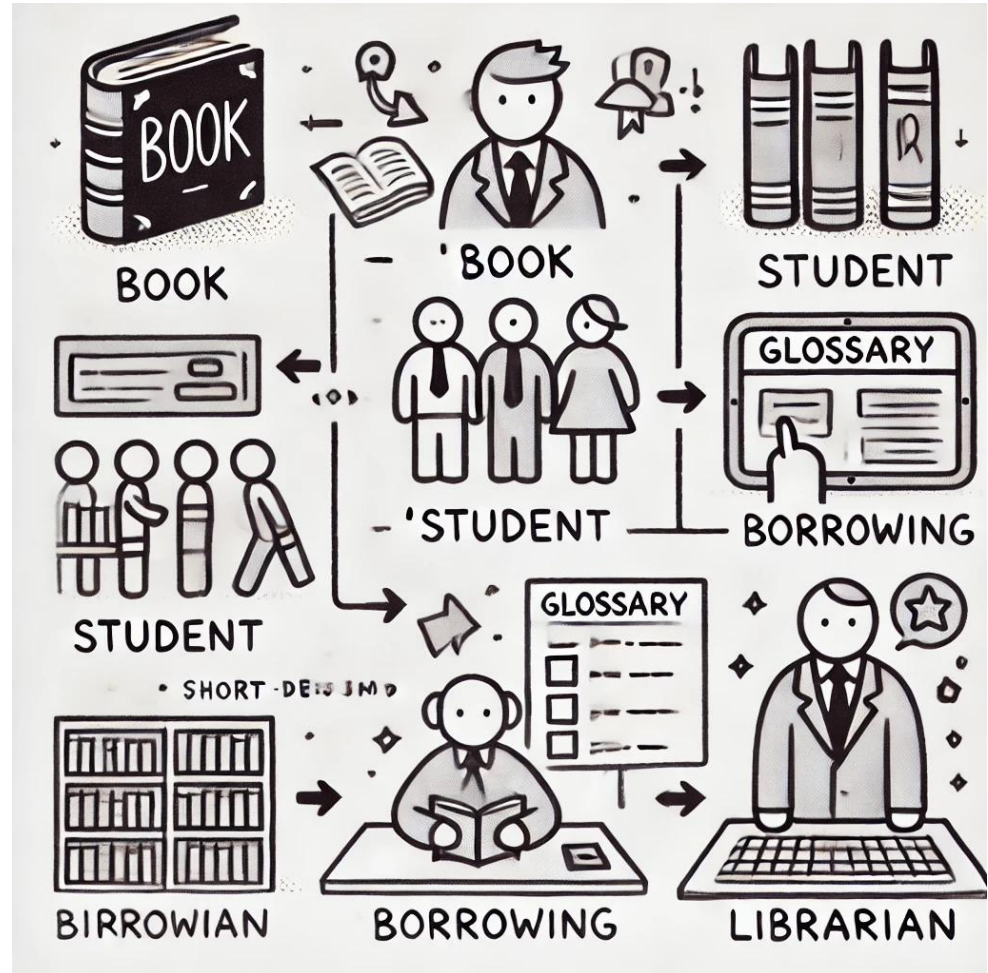


Class



Objects/Instances

Glossary



Glossary: Defining important terms in a system

- A glossary is a **list of terms and definitions that are important** for the system you are analysing.
- It helps everyone involved in the project (developers, stakeholders, users) to **understand key** concepts.



Why is a glossary important?

- **Clarity:** Ensures that everyone understands terms in the same way, **reducing miscommunication**.
- **Consistency:** Provides a **reference point** for terms used throughout the project.
- **Basis for modelling:** Serves as a **basis** for **creating** more detailed system diagrams such as **domains** and class diagrams.

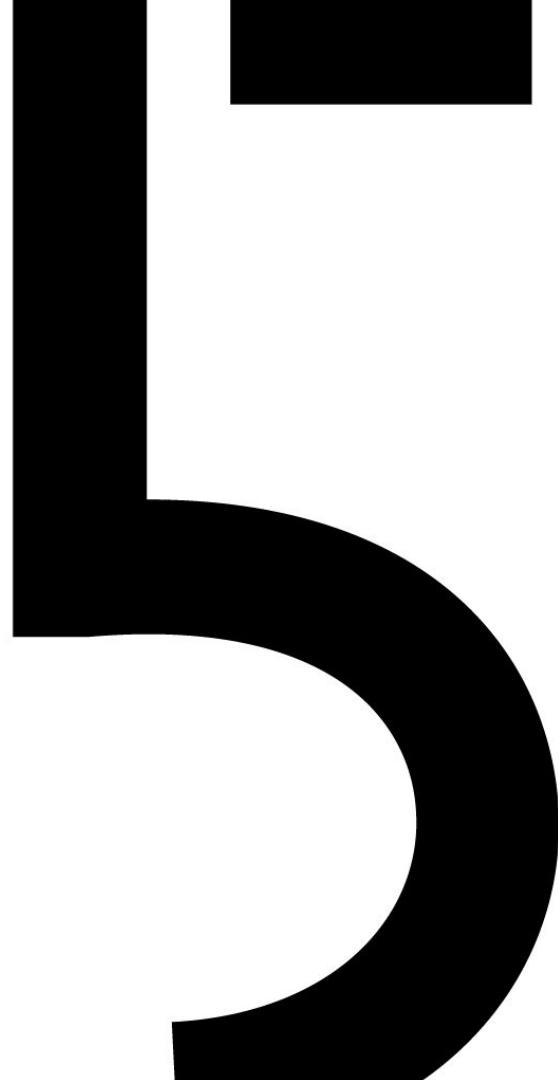
Steps to create a glossary:

- **Define key terms:** Look for **key** concepts and objects in the system (e.g. book, student, order).
- **Define each term: Clearly** explain **the meaning** and **role of** each term in the system.
- **Use consistent language:** Ensure terms are used **consistently** across all documents and diagrams.

Example (Library Management System):

- **Book:** A physical or digital item available for borrowing.
- **Borrowing:** The process of taking out a book for a period of time.
- **Librarian:** A person responsible for managing library resources and helping students with their library needs.
- **Student:** A member of the library who can borrow books.
- **Reservation:** An action to request a book that is not currently available for future use.

Group assignment



Assignment

- Published on canvas (P1 Week 1)

- **Groups of max. 4 =**

Each group member uploads Part 1 (which will be identical to your other 4 members) + Part 2 (unique and individual) $\Rightarrow$ 1 document

Each student uploads their own document on canvas (including 2 parts), **before next class.**

You will automatically be assigned a document for assessment. You assess the document as if you were the user and a software engineer, who paid you a lot of money, provided you with this document. Give all feedback if something is not clear to you or if you need more details.